Lecture 10: traffic light

Goals

- build a stateful circuit according to spec

last time we figured out how we could build a couple real(ish) things out of circuitry

today we're going to do the same for the humble traffic light

all of the circuits for today's discussion are in a single file: traffic_light_system.circ (I will refer to subcircu

let's imagine a 4-way intersection, each direction protected by lights

to simplify the problem, let us decree that the north- and south-facing lights are always the same, and the
are always the same

each traffic light has three bulbs: red, yellow, and green (top to bottom)

each bulb can be told to turn on or to turn off

the "traffic_light" subcircuit is what we're working with

to turn a light on or off, we set the appropriate input to 1 and then toggle the clock (the input at the bottor

the "traffic_light_test" subcircuit uses the former as a subcircuit and adds pretty-colored LEDs to demons

the clock input is controlled by a "push button" component (in the I/O library) that emits a 1 as long as the

first step: figure out what sort of behavior we want

start with N/S red, E/W red

then N/S green, E/W red

then N/S yellow, E/W red

then N/S red, E/W red

then N/S red, E/W green

then N/S red, E/W yellow

then back to beginning

(going red/red is possibly unrealistic, bear with me)

these are states!

we can give them numbers: 0-5

this is different to the stateful combo lock in at least two important ways

we want the lights to DO something different in each state

and we're moving from state to state based on TIME, not input

deal with one problem at a time

now that we've specified the desired behavior, we move down a level of abstraction to consider how we
building blocks we've got

that is: what has to happen in each state to achieve the behavior we want?

0: N/S red-on high, E/W red-on high, N/S green-off high, N/S yellow-off high, E/W green-off high, E/W ye

1: N/S green-on high, N/S red-off high

2: N/S yellow-on high, N/S green-off high

3: N/S red-on high, N/S yellow-off high

4: E/W green-on high, E/W red-off high

5: E/W yellow-on high, E/W green-off high

and now we go another level (of abstraction) deeper

all the on/off signals are controlled by wires, 12 total

(as shown in the "almost_traffic_system" subcircuit)

which wires do we need to turn on at each step to get the desired effect?

thus the wires that need to go high at each state are…

0: pins 0, 3, 5, 6, 9, 11

1: pins 1, 4

2: pins 2, 5

3: pins 0, 3

4: pins 7, 10

5: pins 8, 11

rewrite as binary, then as hex

remember that pin 0 is LEAST SIGNIFICANT BIT (ie, right-most)

0: 1010 0110 1001 -> 0xa69

1: 0000 0001 0010 -> 0x012

2: 0000 0010 0100 -> 0x024

3: 0000 0000 1001 -> 0x009

4: 0100 1000 0000 -> 0x480

5: 1001 0000 0000 -> 0x900

now let's imagine we're storing the state in a register, as we did for the stateful combination lock

in state 0, we want a particular setting of the wires

in state 1, we want a different setting

and so on

do we have a circuit element that lets us choose among different inputs?

yep: mux

(I cheated by using a counter instead of a register to store the state: it's a circuit that just adds to the stor

full implementation shown in the "traffic_system" subcircuit

this also looks a wee bit like the memory we saw previously

storing values and having the mux pick one by its location

the only difference is that we don't have the facility to write new values

there are, in fact, memories exactly like this

they're called ROM: read-only memory

the contents are often literally baked into the computer chips

(old-school video games were often implemented this way, hence the chunk of code you download to rur
called a ROM)

takeaways:

practice building larger/different circuits that do interesting things

state machines are more or less ubiquitous

sometimes we want to move from state to state based on TIME instead of input

encode the thing we want to do as a set of bits, feed those to the combinatorial logic every timestep

these are all remarkably similar to the way "real" computers work

Last modified: 10/05/2026 12:06:14